FemboyEngine 0.2.0-alpha.1 — начало работы

Этот документ описывает, что нужно установить и какие файлы должны лежать рядом с движком, чтобы запустить редактор, рантайм и инструменты разработки на Windows, Linux и macOS, а также какие инструменты нужны для сборки под Android и iOS.

Поддерживаемые платформы

Платформа	Редактор	Рантайм игр	Сборка
Windows 10/11 x64	да	да	clang-cl (основной), GCC 16+ (проверочный)
Linux x64	да	да	Clang 19+, GCC 16+
macOS 13+ (Apple Silicon и Intel)	да	да	Apple Clang из Xcode, Metal-бэкенд
Android 5.0+ (API 21)	нет	да, через SDL3	NDK r28c, Android Gradle Plugin 8.7.3, Gradle 8.12
iOS 11+	нет	да, через SDL3	Xcode 16+, iOS SDK 14.2+

Редактор — настольное приложение. Игры собираются под все перечисленные платформы; мобильные сборки используют платформенный слой SDL3.

Требуемое программное обеспечение

MSVC (компилятор `cl.exe`) не требуется. На Windows проект собирается clang-cl из состава LLVM; MSVC-компилятор не используется. Visual Studio Build Tools нужны только как поставщик Windows SDK и заголовков стандартной библиотеки, на которые опирается clang-cl.

Инструмент	Минимальная версия	Проверенная версия	Зачем
CMake	3.28	3.31.6	конфигурация и сборка
Ninja	1.11	1.12.1	генератор сборки
LLVM / clang-cl	19	23.1.4	основной компилятор Windows
Clang (Linux/macOS)	19 / Xcode 16	—	компилятор Linux и macOS
GCC	15	16.2.0	второй компилятор для проверки переносимости
Python	3.11	3.13	генерация документации и notices
Git	2.40	—	клонирование и версионирование
Vulkan Loader	1.4	1.4	рантайм на Windows и Linux
MoltenVK	1.2	—	Vulkan поверх Metal на

			macOS (либо Metal-бэкенд напрямую)
7-Zip	21	25.01	распаковка LZMA SDK (иначе используется встроенный FbyLZ)
Android Studio	—	NDK r28c	Android SDK, NDK 28.2.13676358, compileSdk 35
OpenJDK	17	17	Gradle-упаковка Android-сборки
Xcode	16	—	сборка под iOS и macOS

Стандарт языка — C++26, поэтому старые компиляторы не подойдут даже при «похожей» версии CMake.

Быстрый старт: скрипты установки

Скрипты лежат в каталоге `SetupScripts/` и разделены по группам ПО. Скрипты «всё сразу» ставят всё; компонентные скрипты — только одну группу.

Группа	**Windows**	**macOS**	**Linux**
C++ toolchain, CMake и зависимости	`SetupScripts\SetupCpp.bat`	`./SetupScripts/SetupCpp.command`	`./SetupScripts/SetupCpp.sh`
Архиватор 7-Zip	`SetupScripts\SetupArchiver.bat`	`./SetupScripts/SetupArchiver.command`	`./SetupScripts/SetupArchiver.sh`
Опциональные SDK DLSS/FidelityFX/XeSS	`SetupScripts\SetupUpscalers.bat`	`./SetupScripts/SetupUpscalers.command`	`./SetupScripts/SetupUpscalers.sh`

Общие точки входа остаются `Setup.bat`, `Setup.command` и `Setup.sh`.

C++-скрипты ставят компилятор, CMake, Ninja, Vulkan-заголовки и закреплённые зависимости. На Windows `SetupCpp` ставит официальный MSI LLVM 23.1.4 и проверяет версию `clang-cl`; Unix-скрипты ставят Clang из пакетного менеджера (19+). Скрипты архиватора ставят 7-Zip. Скрипты апскейлеров только проверяют или скачивают vendor SDK; они опциональны и никогда не входят в репозиторий движка.

Полезные флаги: `--skip-system` — не трогать системные пакеты (образ CI уже содержит инструменты); `--force` — заново скачать C++-архивы. C++-скрипт создаёт `third_party/dependencies.lock` с версиями и хешами архивов. В конце он печатает отчёт о найденных мобильных инструментах: NDK, JDK и Xcode.

Из архива GitHub

1. Открыть страницу репозитория и выбрать **Code > Download ZIP**.
2. Распаковать архив в каталог без пробелов и не-ASCII символов.
3. Запустить `SetupCpp` и `SetupArchiver` для своей ОС.
4. При необходимости запустить `SetupUpscalers` и включить vendor backend.
5. Настроить и собрать:

```
cmake --preset dev-clang
cmake --build --preset dev-clang
ctest --preset dev-clang --output-on-failure
```

В архиве нет vendor SDK. C++-скрипт ставит toolchain и раскладывает

закреплённые исходники; archiver-скрипт ставит 7-Zip. Скрипты апскейлеров

нужны только при явном включении DLSS/FidelityFX. Без них FSR1, core-путь

FSR2 и собственная генерация кадров собираются и работают с runtime-fallback.

Ручная установка

Если скрипт запускать не хочется, набор действий одинаков: поставить компилятор, CMake, Ninja, Vulkan-заголовки и разложить сторонние библиотеки.

Windows

6. Установить LLVM 23.1.4 (clang-cl) и добавить `bin` в `PATH`.
7. Установить Visual Studio Build Tools с рабочей нагрузкой «C++ Build Tools» (Windows SDK + заголовки стандартной библиотеки). Компилятор MSVC при этом не используется.
8. Установить CMake 3.31 и Ninja 1.12 (можно из поставки Build Tools).
9. Установить Vulkan SDK 1.4 — он даёт `vulkan-1.dll` и `VULKAN_SDK`.
10. Установить 7-Zip (по желанию).
11. Скачать архивы, которые обычно качает скрипт: SDL3 `release-3.4.18`, Vulkan-Headers `v1.4.366`, glslang `15.3.0`, miniaudio `0.11.23`, Box2D `v3.2.0`, RNNoise `0.1.1`, Monocypher `4.0.2` и распаковать их в те же каталоги `third_party/`, что создаёт скрипт. HarfBuzz и SheenBidi скачивать не нужно: они уже лежат в репозитории.

Linux

```
sudo apt install build-essential clang cmake ninja-build git pkg-config libvulkan-dev
vulkan-tools p7zip-full
```

Дополнительно раскладываются те же архивы, что и на Windows. GCC 16 ставится из дистрибутива или собирается отдельно; для основной сборки достаточно Clang.

macOS

```
xcode-select --install
brew install cmake ninja git p7zip glslang vulkan-headers molten-vk llvm
```

Xcode Command Line Tools обязательны: они дают `clang++`, Metal-фреймворки и заголовки. Для Metal-бэкенда Vulkan SDK не нужен.

Сборка

Из корня репозитория:

```
cmake -S . -B build-dev -G Ninja -DCMAKE_BUILD_TYPE=RelWithDebInfo -DFBY_BUILD_TESTS=ON
cmake --build build-dev
ctest --test-dir build-dev --output-on-failure
```

На Windows те же команды выполняются в окружении `vcvars64`; зависимости и инструменты подготавливает `SetupScripts/SetupWindows.ps1`. Основные опции CMake: `FBY_BUILD_EDITOR`, `FBY_BUILD_TESTS`, `FBY_BUILD_TOOLS`, `FBY_BUILD_SANDBOX`, `FBY_BUILD_SERVER`, `FBY_BUILD_LOBBY`, `FBY_WITH_ASAN`, `FBY_RHI_BACKEND` (`auto`, `vulkan`, `metal`, `null`).

Готовые исполняемые файлы раскладываются по назначению:

Каталог	Что внутри
`build-dev/bin`	приложения: `fby_editor`, `fby_sandbox`, `fby_server`, `fby_lobby`, плагины платформ и `plugins/`
`build-dev/bin/tools`	консольные инструменты: `fby_fontc`, `fby_textc`, `fby_fempak`, `fby_scene_tool`, `fby_template_gen`, `fby_aotc`, `fby_loc_tool`, `fby_shaderc`
`build-dev/bin/tests`	тестовые исполняемые файлы (`fby_*_test`), их запускает `ctest`
`build-dev/bin/lib`	статические библиотеки
`build-dev/bin/plugins`	нативные плагины движка

Файлы `*.pdb` рядом с исполняемыми файлами — это отладочные символы, которые создаёт компилятор. Они нужны только отладчику и не требуются для запуска; в релизной поставке их можно не распространять.

Редактор кода и IDE

Собственный текстовый редактор движку не нужен. Для разработки используется внешний редактор, который умеет читать CMake-проект и `compile_commands.json`.

Рекомендуемый вариант:

- **VS Code**;
- расширение **clangd**;
- расширение **CMake Tools**;
- **CodeLLDB** или `cppdbg` для отладки;
- RenderDoc для отладки GPU.

CLion тоже подходит: он нативно понимает CMake и умеет запускать `fby_editor` и тесты. Visual Studio можно использовать как IDE, но не как MSVC-сборку; подробности ниже.

VS Code: настройка

12. Установить VS Code.
13. Установить расширения:

- `llvm-vs-code-extensions.vscode-clangd`;
- `ms-vscode.cmake-tools`;
- `vadimcn.vscode-lldb` либо `ms-vscode.cpptools` для `cppdbg`.

14. Открыть корень репозитория (папку, в которой лежит `CMakePresets.json`).
15. Отключить встроенный IntelliSense, иначе он будет конфликтовать с clangd:

```
{
  "C_Cpp.intelliSenseEngine": "disabled"
}
```

16. Выбрать CMake-пресет `dev-clang` или `dev-clangcl`.
17. Сконфигурировать проект:

```
cmake --preset dev-clang
```

18. Собрать:

```
cmake --build --preset dev-clang
```

19. Запустить тесты:

```
ctest --preset dev-clang
```

В репозитории уже лежат:

- `CMakePresets.json` — пресеты;
- `.clangd` — путь к базе компиляции;
- `.vscode/settings.json` — настройки clangd и CMake Tools;
- `.vscode/launch.json` — запуск редактора и `frame_composer` теста.

clangd

clangd должен использовать тот же Clang, которым собирается проект. В настройках:

```
{
  "clangd.arguments": [
    "--compile-commands-dir=${workspaceFolder}/build-audit",
    "--query-driver=C:/Users/<user>/AppData/Local/Programs/LLVM/bin/clang++.exe"
  ]
}
```

`--query-driver` нужен, чтобы clangd нашёл системные заголовки Windows SDK. Если используется другой build-каталог, заменить путь в `--compile-commands-dir` или в `.clangd`.

Нельзя держать одновременно включёнными C/C++ IntelliSense и clangd: получится два разных набора диагностик и переходов.

Пресеты сборки

Пресет	Компилятор	Назначение
`dev-clang`	`clang++`	Debug, тесты, `compile_commands.json`
`release-clang`	`clang++`	RelWithDebInfo, тесты
`dev-clangcl`	`clang-cl`	Debug через проверенный Windows-toolchain
`audit-clang`	`clang++`	существующий `build-audit` для clangd

Разные компиляторы должны использовать **разные build-каталоги**. Нельзя переконфигурировать `build-audit` с clang++ на clang-cl: CMakeCache останется от старого toolchain.

Отладка

Для `cppdbg` используется gdb из MinGW/LLVM либо CodeLLDB с LLVM. Пример запуска:

```
{
  "name": "Editor (Debug)",
  "type": "cppdbg",
  "request": "launch",
  "program": "${workspaceFolder}/build-dev/bin/fby_editor.exe",
  "cwd": "${workspaceFolder}",
  "MIMode": "gdb",
  "miDebuggerPath": "gdb.exe"
}
```

Для GPU-отладки использовать RenderDoc: он подключается к `fby_editor.exe` и снимает кадр без встраивания отладчика в редактор.

Поддержка MSVC

Сейчас — нет. Проект сознательно собран вокруг Clang:

- C++26 и строгие флаги проверяются на Clang 23.1.4;
- clang-cl уже даёт Windows ABI, Windows SDK и MSVC-совместимый драйвер;
- MSVC-компилятор (`cl.exe`) не требуется;
- добавление MSVC означает второй набор флагов, предупреждений, поведения STL и C++26-фич, а также отдельную CI-матрицу.

Это не значит, что MSVC «запрещён навсегда». Если он нужен как **необязательный второй компилятор**, делать это нужно отдельной задачей:

20. добавить пресет `dev-msvc`;
21. проверить C++26-фичи, используемые движком;
22. добавить `FBY_COMPILER_MSVC` для различий флагов;

23. добавить отдельный CI-job;
24. не заменять Clang, а держать MSVC как дополнительную проверку.

До этого основной и проверенный путь — **Clang / clang-cl**. Visual Studio можно использовать как IDE с toolchain clang-cl, но не как замену сборки на MSVC.

Запуск редактора

Редактор открывает проект из **текущего рабочего каталога**: он ищет `project.femproj`, поднимает виртуальную файловую систему на `assets/`, пишет настройки проекта в `editor.ini` рядом с `project.femproj`, а раскладку ImGui хранит в `Saved/Config/editor.ini`. Поэтому запускать его нужно из корня проекта (или из корня репозитория для демонстрационного проекта).

```
cd %USERPROFILE%\FemboyEngine
build-dev\bin\fby_editor.exe
```

Linux и macOS:

```
cd /path/to/project
./build-dev/bin/fby_editor
```

Все изменяемые файлы движок складывает в `Saved/` в рабочем каталоге, поэтому каталог проекта остаётся чистым:

Путь	Содержимое
`Saved/Logs/<категория>/<категория>-YYYYMMDD-HHMMSS.log`	полный лог запуска: `engine`, `editor`, `sandbox`, `server`, `lobby`. Хранится 20 последних файлов на категорию, более старые удаляются автоматически
`Saved/Crashes/<YYYYMMDD-HHMMSS>-<pid>/report.txt`	отчёт о падении: время, приложение, pid, причина, адрес и, на Linux и macOS, стек вызовов
`Saved/Crashes/<...>/crash.dmp`	минидамп Windows для отладчика
`Saved/Config/editor.ini`	настройки и раскладка панелей редактора
`Saved/Cache/`	кэш промежуточных данных

Переменные окружения: `FBY_STORAGE_DIR` переносит корень `Saved/` (например на диск с местом), `FBY_LOG_FILE=0` отключает запись лога в файл (консоль продолжает работать).

Что должно лежать рядом

Путь	Обязателен	Назначение
`build-dev/bin/fby_editor`	да	сам редактор
`assets/`	да	шрифты, локализация, сцены, атласы, конфиги проекта
`assets/fonts/IBMPlexSans-Regular.ttf`, `...-Bold.ttf`, `...-Italic.ttf`, `...-BoldItalic.ttf`, `IBMPlexMono-Regular.ttf`	да	семейство шрифтов редактора и текста сцены
`assets/fonts/IBMPlex.family.toml`	да	манифест семейства: пять слотов стилей
`assets/fonts/Inter.fbyfont`	да	испечённый атлас для

		устаревшего пути UI
`assets/loc/ui.fbin`	да	таблица локализации интерфейса
`assets/scenes/*.toml`	да	сцены проекта
`project.femproj`	да	описание проекта: сцена по умолчанию, симуляция, рендер, доступность
`build-dev/`	да	дерево сборки CMake: там лежат исполняемые файлы и результаты сборки
`build-dev/bin/fby_sandbox`	нет	второй процесс для режима Play-in-Editor по сети
`Saved/`	создаётся	логи, отчёты о падениях, кэш и настройки редактора
`Saved/Config/editor.ini`	создаётся	раскладка панелей и настройки редактора для этой машины
`build-dev/bin/plugins/`	нет	нативные плагины движка
Vulkan Loader в системе	да	окно и GPU-рендер на Windows и Linux
`Projects/`	нет	каталог по умолчанию для новых проектов

Выбор проекта

Проект — это каталог с файлом `project.femproj` и подкаталогами `assets/`, `configs/`. Новые проекты создаются в `Projects/` внутри корня движка; открыть существующий можно через дашборд редактора по пути к `project.femproj` или к папке проекта. Отдельного аргумента командной строки для проекта нет: проект определяется рабочим каталогом, поэтому «ярлык на проект» — это ярлык с другим рабочим каталогом.

Инструменты разработки

Кроме редактора в `build-dev/bin` собираются отдельные исполняемые файлы. Все они — обычные консольные программы, читают файлы и не требуют GUI.

Инструмент	**Назначение**	**Что нужно из файлов**
`fby_editor`	редактор: сцены, ассеты, PIE, предпросмотр текста	`assets/`, `project.femproj`
`fby_sandbox`	песочница и клиент Play-in-Editor	`assets/`
`fby_server`	выделенный сервер	каталог проекта по желанию
`fby_lobby`	сервис лобби и матчмейкинга	не требуется
`fby_fontc`	сборка `.fbyfont` из TTF/OTF	TTF-файл на входе
`fby_textc`	компиляция размеченного текста в `.fbin`	`.toml` текста на входе
`fby_fempak`	упаковка ассетов в `.fempak`	каталог ассетов
`fby_scene_tool`	проверка и конвертация сцен	`.toml` сцены
`fby_template_gen`	создание каркаса нового проекта	каталог назначения
`fby_aotc`	нативизация скриптов (AOT)	проект с байткодом
`fby_loc_tool`	проверка и сборка таблиц локализации	`.toml` локализации
`fby_shaderc`	компиляция шейдеров в SPIR-V	`.vert`/`.frag`; вызывается сборкой автоматически

Инструменты лежат в `build-dev/bin/tools`, тестовые исполняемые файлы — в `build-dev/bin/tests`.

Сборка и тесты запускаются через пресеты CMake; установка зависимостей — в `SetupScripts/*`.

Ярлыки для быстрого запуска

Рабочий каталог — единственное, что отличает запуск «движка» от запуска «проекта», поэтому ярлык всегда должен задавать его явно.

Windows

25. Правый щелчок по `build-dev\bin\fby_editor.exe` → «Отправить» → «Рабочий стол (создать ярлык)».
26. Свойства ярлыка: «Рабочая папка» = корень проекта (например `%USERPROFILE%\FemboyEngine`), «Объект» = полный путь к `fby_editor.exe`.
27. Переименовать ярлык в «FemboyEngine» и перетащить на панель задач или в меню «Пуск».
28. Для отдельного проекта сделать второй ярлык с тем же «Объектом», но «Рабочей папкой» = папка проекта. Это и есть «выбор проекта» без диалога.

macOS

29. Открыть папку проекта в Finder.
30. Перетащить `fby_editor` (или `.app`, если собран бандл) в Dock, удерживая `Command`.
31. Для конкретного проекта создать `.command`-файл с двумя строками: `cd /path/to/project` и `/path/to/fby_editor`; Finder запускает его двойным щелчком, а ярлык можно положить в Dock.
32. `SetupScripts/Setup.command` запускается так же — двойным щелчком.

Linux

Файл `~/.local/share/applications/fby-editor.desktop`:

```
[Desktop Entry]
Type=Application
Name=FemboyEngine
Exec=/path/to/fby_editor %f
Path=/path/to/project
Icon=/path/to/Logo.png
Terminal=false
Categories=Development;
```

После сохранения — «Закрепить» в доке или на панели. Второй `.desktop` с другим `Path` даёт ярлык на другой проект.

Android и iOS

Мобильные сборки используют платформенный слой SDL3: окно, ввод и жизненный цикл приложения берутся из SDL, рендер — Vulkan (Android) и Metal/Vulkan через MoltenVK (iOS). Редактор на мобильные платформы не собирается.

Android: Android Studio, NDK `28.2.13676358` (r28c), OpenJDK 17, Gradle 8.12, Android Gradle Plugin 8.7.3, `compileSdk 35`, минимальный уровень API 21. Конфигурация CMake для Android выполняется через toolchain-файл NDK, упаковка — Gradle-проектом SDL3.

iOS: macOS с Xcode 16 или новее, iOS SDK 14.2+, минимальная целевая версия iOS 11.0. Конфигурация CMake выполняется с `-DCMAKE_SYSTEM_NAME=iOS`, подпись и упаковка — средствами Xcode.

Точный список версий всегда можно получить двумя командами:

```
cmake --version
ninja --version
```

и, для мобильных, `ls "$ANDROID_NDK_HOME/source.properties"` и `xcodebuild -version`.

Типичные проблемы

Симптом	Причина	Решение
Редактор не видит ассеты	запущен не из корня проекта	запустить из каталога с `assets/` и `project.femproj`
Ошибка Vulkan при старте	нет Vulkan Loader в системе	установить Vulkan Runtime или собрать с `-DFBY_RHI_BACKEND=null`
Редактор или игра закрылись без сообщения	падение процесса	посмотреть `Saved/Crashes/` в рабочем каталоге: `report.txt` и `crash.dmp`
Нужен лог запуска для отчёта	логи теперь в файле	взять последний файл из `Saved/Logs/<категория>/`
`glslang` не найден при сборке	не разложен `third_party/glslang-src`	запустить скрипт установки заново
Нет шрифтов в редакторе	отсутствуют TTF или манифест семейства	вернуть `assets/fonts/` из репозитория
Ошибка CMake про harfbuzz или SheenBidi	повреждён или удалён каталог `third_party/`	восстановить `third_party/harfbuzz-download/` и `third_party/sheenbidi-download/` из репозитория
Android-сборка не находит NDK	не задан `ANDROID_NDK_HOME`	указать путь к NDK r28c и перезапустить конфигурацию
Windows-сборка падает на заголовках	нет Windows SDK	установить Build Tools с рабочей нагрузкой C++